

A Report On

MUSIC GENRE CLASSIFICATION

SUBMITTED TO
THE DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

for the partial fulfilment of the award of degree of
B.TECH. (COMPUTER SCIENCE & ENGINEERING)
OR
LOVELY PROFESSIONAL UNIVERSITY, PHAGWARA

SUBMITTED BY

NAME: GAURAV KUMAR
REG.NO: 12017195
SESSION: 2020-2024

COLLEGE

LOVELY PROFESSIONAL UNIVERSITY, PHAGWARA

STUDENT'S DECLARATION

We hereby certify that the work which is being done in the project entitled “**Music Genre Classification**” by “**GAURAV KUMAR**, as per course requirement of **B.Tech CSE** submitted in the Department of Computer Science & Engineering at **COLLEGES**, under **UNIVERSITY** is record of our own work carried out under the supervision of “Dr. Sita Saini”. The matter presented in this report has not been submitted by us in any other institute for the award of B.Tech Degree

Gaurav Kumar

Introduction

The aim of this project was to compare machine learning algorithms in their ability to automatically classify song excerpts into the correct musical genre. For humans who are familiar with the genres in question, music genre recognition (MGR) is not an especially difficult task, as exemplified in Gjerdingen (2008). Most people wellacquainted with Western popular music are able to identify that a given Metallica song is an example of the heavy metal genre, and that a given Status Quo song is an example of rock music, just by listening to the audio. Importantly, they are usually able to perform this type of broad genre classification even if they are unfamiliar with the song or artist in question: ‘heard’ qualities of the sound are normally enough. As Gjerdingen (ibid.) describes, only a short sample of audio is usually needed, sometimes less than a second. A more fine-grained classification into musical subgenre usually requires some level of expertise on the part of the listener, and the greater the level of expertise, the more accurate the classification that can be made. The question is raised as to whether software can be written to perform genre classifications as well as humans, and what type of approaches work best. Despite being an interesting endeavour in and of itself, there are practical, realworld applications of automated music genre recognition, with music streaming services representing the most pertinent example. Pandora is a US-based streaming and recommendation platform that employs musicologists to annotate songs with genre and rhythm features, with the aim of creating better recommendation playlists (Lawton, 2017, ‘Deconstructing the recommendation engine’). Users may want to discover tracks that are similar to those that they are already a fan of, and it makes sense for providers to have such metadata for their songs available, in order to facilitate this kind of discovery. Given the vast and ever-growing quantity of music available on the internet, and the need for services to manage increasingly large song databases, manual analysis and annotation of individual songs does not seem like a sustainable long-term solution. From an engineering perspective, despite the cost of developing and maintaining the system, long-term expenditures could be significantly reduced with the use of accurate, automated genre-recognition systems. Large streaming services are surely already aware of the prospects of automated genre recognition techniques, but questions remain as to the best

way a music genre recognition system can be architected. Various machine learning techniques have been applied to problems - including genre recognition - in the field of Music Information Retrieval. A range of possible approaches toward the feature engineering of such projects exist. In general, machine learning classifiers will perform well when they are able to 'understand' what makes a data sample a member of a particular class, and, in many cases, when classes tend to be easily 'separable' from one another (i.e. with different classes varying significantly in terms of their average feature values). When humans are able to recognize the genre of a given audio track, they typically do so based on 'high-level' aspects of the sound in question, e.g., the instruments in use, the feel with which they are played, the 'overall sound' of the song, etc. These types of emergent musical property may be difficult for software to recognize; however, given the direct availability of audio files, it is possible for software to detect aspects of audio files that humans cannot perceive through mere listening alone. Automatic genre classification based on these types of features has already been performed, with promising results. The general motivation behind such approaches is that different musical genres tend to use different types of sounds, perceptible by humans and potentially detectable by the right software as well. This study compares several machine learning algorithms in their ability to classify songs accurately. First, a survey of existing literature is carried out, looking at how machine learning has been applied to problems of music classification and audio classification in general. Tools for extracting features from audio files are discussed, as well as techniques for classifying music based on other modalities such as song lyrics and album artwork. This is followed by a discussion of machine learning techniques fit for present purposes. Various candidate datasets are evaluated, with a justification of the final dataset chosen and an explanation of how it was preprocessed and reduced in size. An explanation of the features extracted, and an analysis of the features chosen (in terms of usefulness), are then presented. After this, a description of the machine learning models follows, as well as a presentation of the results of each classifier. Finally, an interpretation of the results is carried out, in addition to an evaluation of design choices and an assessment of the project management. The study is concluded with the main lessons learned, and recommendations for future research.

Related Work

Automatic genre classification of music can be attempted in a variety of ways. A typical approach involves processing a dataset of audio files, extracting features from them, and then using a dataset of these extracted features to train a machine learning classifier. This is the strategy that the majority of this literature review will focus on, as it encompasses the techniques that have proven to be most effective when classifying via one modality; as such, it is the approach that has been studied most extensively. However, there exist other modalities to make use of, including song lyrics, album reviews, and album artwork. Each genre of music tends to produce content with distinct approaches to all aspects of the finished product, and, as a result, genre classification via analysis of related text and imagery has also proven fairly successful. ‘Multimodal’ classification has shown some promise, though more research is needed. After exploring these different approaches, surveying the existing literature in this area, and justifying the choice of modality that this particular study focuses on, this section will look more carefully at the different A.I. techniques that classification systems (including those in the field of music genre recognition) make use of. The majority of approaches involve some kind of machine learning classifier, such as an artificial neural network or a support-vector machine, although different strategies are sometimes used in text classification. This section will also justify the types of classifier chosen for this particular project.

METHOD:

1.Dataset selection:

The choice of dataset plays an important role in any machine learning project. Of ten times, there exists a number of potentially suitable datasets available, each with their own advantages and disadvantages, and the decision to choose one dataset over another can significantly influence the course of the project. This section will outline the primary options considered for this study, and a justification of the final choice made.

2.GTZAN Dataset:

The GTZAN dataset, detailed in Tzanetakis (2002), is a relatively well-known resource in the field of MIR. It consists of 10 groups of 100 30-second song excerpts, with each group representing one genre, totalling 1000 audio files overall. The dataset comes pre-organised into folders - one per genre - making it fairly straightforward to navigate, and the balanced nature of the dataset makes it appealing for machine learning projects. However, despite these attractive features, GTZAN bears a number of disadvantages. First, with only 100 items per class, it is a relatively small dataset, in the context of other machine learning projects. Deep learning is often considered to thrive on large amounts of data (Ng, 2015), and having fewer training examples means that the classifiers have less information from which to build their models. It therefore made sense to see whether a larger dataset would be available.

Further, Sturm (2013) has identified a number of issues with the dataset, including repetitions of songs (i.e. different excerpts of the same song appearing as different tracks in the dataset) and mislabelling of tracks (i.e. tracks being labelled as the wrong genre). For these reasons, GTZAN was considered to be unsuitable for this project.

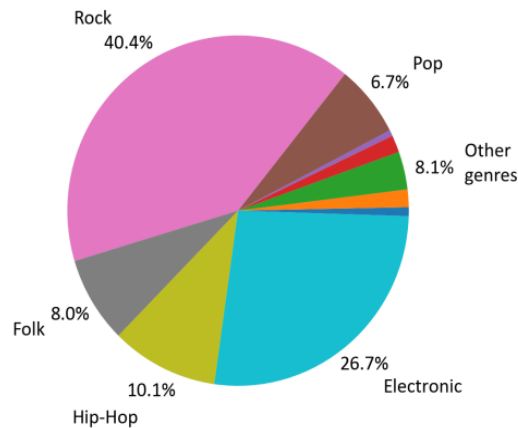
The Million Song Dataset:

The second dataset considered was the Million Song Dataset (Bertin-Mahieux et al., 2011). As implied by the name, the MSD consists of one million songs, spreading over a variety of genres and subgenres. One million data points is a large dataset, and many good machine learning projects have been undertaken using far fewer training examples. However, the MSD differs from other datasets such as GTZAN in a significant way: while GTZAN is able to provide access to the audio files of the dataset themselves (due to the songs' relative obscurity), the MSD deals with contemporary popular music, meaning that access to audio files cannot be provided, on grounds of copyright. A dataset of a million audio files would also take up several terabytes of data, making it difficult to download and store for many researchers. Instead, the dataset consists of pre-extracted features and metadata, such as danceability and popularity, for each of the songs. While these may be interesting to analyse in an other context, it was deemed necessary to be able to have access to the audio files themselves, in order to allow for extraction and analysis of audio features such as the timbre and pitch-content features detailed above. Therefore, another dataset was required.

Data Preprocessing:

Preliminaries:

several stages of pre-processing were required in order to ready the dataset for classification (note: the changes described in this and the next paragraph were made with Microsoft Excel, not with code). First, 'tracks.csv' was downloaded from the FMA webpage (Deffieux et al., 2017b). This provided information on the genres associated with each song. Lines which contained no information in the 'genre top' field were deleted, as some tracks were not given a 'top genre', but instead were given a list of genres in another column. This was done as part of clarifying the problem, in order to discount songs that were not given a single genre and had apparently been marked to suggest that they may have been classifiable into more than one. After this, several genres were removed from the dataset. 'Spoken' tracks were deleted because spoken-word poetry does not really constitute music, at least with regard to the particular project at hand. 'Experimental' tracks were deleted because experimental music is broad and diverse, and songs under the label of 'experimental' may bear little sonic similarity to one another, making the genre hard to define and potentially making the tracks very difficult to classify. 'Instrumental' tracks were deleted because a song from any genre can be made instrumental simply by removing the vocals, meaning that many tracks under the label may bear little similarity to one another (arguably, 'instrumental' is not even a genre at all). Finally, 'international' music was discounted because again, there may be too much diversity under such a broad label.

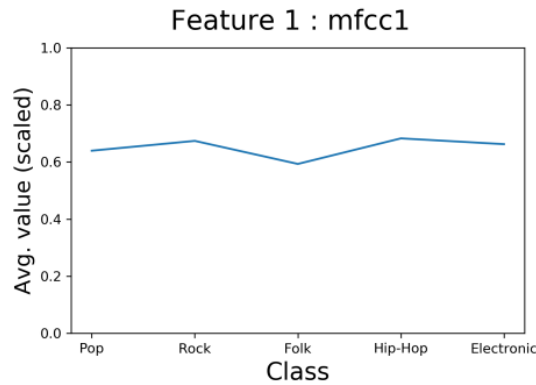


Genre and Feature Analysis:

At this point, prior to the classification phase of the project, two stages of analysis were performed. The first involved plotting the average value for each class, for each feature extracted. The second involved comparing and ranking each of the features by their F-value, to see which would be most relevant in classification.

Genre Analysis:

In order to compare classes by their average value for each feature, the standard dataset csv was used, as it made sense to inspect only non-synthetic data. The average values were computed in the following way (rough pseudocode representation): 1. For each feature f : (a) For each genre g : i. Add up all g 's instances, in terms of f ii. Divide by the number of instances in g (in this case, 2331) (b) Plot the results. The final graphs are viewable at a link given in the appendix, with an example graph viewable in figure 3.2. This graph plots the average values for the MFCC1 feature, which is the MFCC that deals with the lowest frequency range. For this feature, the value for Folk is the lowest, whereas the value for Hip-Hop is the highest. It might be supposed that Folk's position as the lowest value may be due to it being generally quieter than the other genres listed here, meaning that lower values would be expected for each of the MFCCs. However, for some of the other MFCCs, such as MFCC6, Folk's average value was actually the highest of all genres. This seems to suggest that the different genres activated different frequency ranges to different degrees: genres like Pop may have had more activation in the lowest frequencies, suggesting a bossier mix, whereas Folk seems to have had more activation in the middle frequencies (e.g. those dealt with by MFCCs 6 to 9), suggesting a mix with greater emphasis on the Mids. This chimes with what we know about the instruments often used for each of these genres: Folk music that consists of just acoustic guitar and vocals will not be as bossy as Pop music played with a loud, thudding kick drum and synthesized bassline. A visual scan of the graphs seems to suggest that Folk often has quite different values from the other classes - for many features, it has the lowest value, but for



Results

Here we see the results of the project:

Genres The average genre scores across the standard dataset were as follows: Electronic: 60%, Folk: 73%, Hip-Hop: 74%, Pop: 41%, Rock: 73%. Although performance varied with each classification algorithm, similar trends were identified in the results of each. For example, across most experiments, Folk and Hip-Hop were classified the most accurately. Folk may have been one of the most successfully classified genres because, as discussed earlier, its average scores for a number of the features were significantly different to those of other genres (refer to the MFCC1 diagram in the 'Feature and Genre Analysis' section of this report for an example). This implies the presence of sonic differences that were detected successfully in the training of the algorithms. Hip Hop's place as the most accurately classified genre is harder to explain; it may be that the presence of rapped vocal passages (i.e. vocal passages that are not sung) helped differentiate it, if such passages led to distinctive feature trends. Further investigation of the dataset would be required to understand why Hip-Hop was classified the best. Electronic tracks were misclassified as Hip-Hop more than they were misclassified as any other genre, and vice-versa. Given the nature of the two genres, this makes sense: Hip-Hop beats tend to be made using electronic samples such as those of the Roland R-808 drum machine, and the two genres bear much similarity in terms of production. Instrumentally, Hip-Hop typically bears very little in common with Folk music - the latter tends to be based around acoustic and electric guitars - and this explains why there was very little confusion between the genres, less than 5% in all but one experiment. Across all experiments, Pop was misclassified more than any other genre by a significant margin. Although accuracy was notably higher than random chance (20%), its final average accuracy was nearly half of that of the best-classified genre (Folk). It is unclear exactly why this is the case. A potential explanation may lie in the fact that as a concept, 'Pop' music may be more broadly and ambiguously defined than the other genres in the dataset. To take two contemporary examples, Ed Sheeran's music is often thought of as Pop, despite being mostly guitar-based.

Here is the requirements and the data of the code

Dataset name: GTZAN Dataset

Modules Required:

Python search Features, Scipy.io.Wavfile, Os, Pickle, Random, Operator, Math,etc.

CODE:

💡 Importing libraries:

```
from python_speech_features import mfcc
import scipy.io.wavfile as wav
import numpy as np

from tempfile import TemporaryFile

import os
import pickle
import random
import operator

import math
```

[37] ✓ 0.9s Python

:

Function to get difference between vectors:

```
# function to get the distance between feature vecotrs and find neighbors
def getNeighbors(trainingSet, instance, k):
    distances = []
    for x in range(len(trainingSet)):
        dist = distance(trainingSet[x], instance, k) + distance(instance, trainingSet[x], k)
        distances.append((trainingSet[x][2], dist))

    distances.sort(key=operator.itemgetter(1))
    neighbors = []
    for x in range(k):
        neighbors.append(distances[x][0])

    return neighbors
```

[38] ✓ 0.5s Python

Identifying the class of instance:

```
# identify the class of the instance
def nearestClass(neighbors):
    classVote = {}

    for x in range(len(neighbors)):
        response = neighbors[x]
        if response in classVote:
            classVote[response] += 1
        else:
            classVote[response] = 1

    sorter = sorted(classVote.items(), key = operator.itemgetter(1), reverse=True)

    return sorter[0][0]
```

[39] ✓ 0.8s Python

Function To Evaluate the Model:

```
# function to evaluate the model
def getAccuracy(testSet, prediction):
    correct = 0
    for x in range(len(testSet)):
        if testSet[x][-1] == prediction[x]:
            correct += 1
    return (1.0 * correct) / len(testSet)
```

[40] ✓ 0.7s Python

Directory that holds the wav files:

```
# directory that holds the wav files
directory = "C:/Users/hp/OneDrive/Documents/Projects/Music genre classifier/Data/genres_original/"
# binary file where we will collect all the features extracted using mfcc (Mel Frequency Cepstral Coefficients)
f = open("my.dat", 'wb')

i = 0
for folder in os.listdir(directory):
    i += 1
    if i == 11:
        break
    for file in os.listdir(directory+folder):
        try:
            (rate, sig) = wav.read(directory+folder+"/"+file)
            mfcc_feat = mfcc(sig, rate, winlen=0.020, appendEnergy=False)
            covariance = np.cov(np.matrix.transpose(mfcc_feat))
            mean_matrix = mfcc_feat.mean(0)
            feature = (mean_matrix, covariance, i)
            pickle.dump(feature, f)
        except Exception as e:
            print('Got an exception: ', e, ' in folder: ', folder, ' filename: ', file)

f.close()
```

[41] ✓ 1m 30.4s Python

... Got an exception: File format b'\xcb\x15\x1e\x16' not understood. Only 'RIFF' and 'RIFX' supported. in folder: jazz
filename: jazz.00054.wav

Splitting Dataset into the training and testing respectively:

```
# Split the dataset into training and testing sets respectively
dataset = []

def loadDataset(filename, split, trSet, teSet):
    with open('my.dat', 'rb') as f:
        while True:
            try:
                dataset.append(pickle.load(f))
            except EOFError:
                f.close()
                break
        for x in range(len(dataset)):
            if random.random() < split:
                trSet.append(dataset[x])
            else:
                teSet.append(dataset[x])
    trainingSet = []
    testSet = []
    loadDataset('my.dat', 0.66, trainingSet, testSet)
```

[42] ✓ 0.1s Python

```
def distance(instance1 , instance2 , k ):
    distance = 0
    mm1 = instance1[0]
    cm1 = instance1[1]
    mm2 = instance2[0]
    cm2 = instance2[1]
    distance = np.trace(np.dot(np.linalg.inv(cm2), cm1))
    distance += (np.dot(np.dot((mm2-mm1).transpose(), np.linalg.inv(cm2)), mm2-mm1 ))
    distance += np.log(np.linalg.det(cm2)) - np.log(np.linalg.det(cm1))
    distance -= k
    return distance
```

[43] ✓ 0.1s Python

Making Prediction's using KNN:

```
# making predictions using KNN
leng = len(testSet)
predictions = []
for x in range(leng):
    predictions.append(nearestClass(getNeighbors(trainingSet, testSet[x], 5)))

accuracy1 = getAccuracy(testSet, predictions)
print(accuracy1)
```

[44] ✓ 6m 24.6s Python

... 0.6991404011461319

```
from collections import defaultdict
results = defaultdict(int)

directory = "C:/Users/hp/OneDrive/Documents/Projects/Music genre classifier/Data/genres_original"
i = 1
for folder in os.listdir(directory):
    results[i] = folder
    i += 1
```

[47] ✓ 0.6s Python

Testing some sample data:

```
# testing the code with external samples
|
test_dir = "https://uweb.engr.arizona.edu/~429rns/audiofiles/audiofiles.html/"
# test_file = test_dir + "test.wav"
test_file = test_dir + "test2.wav"
# test_file = test_dir + "test4.wav"
```

[56] ✓ 0.1s Python



```
(rate, sig) = wav.read(test_file)
mfcc_feat = mfcc(sig, rate, winlen=0.020, appendEnergy=False)
covariance = np.cov(np.matrix.transpose(mfcc_feat))
mean_matrix = mfcc_feat.mean(0)
feature = (mean_matrix, covariance, i)
```



Thank You!
